7

调度：简介

到现在为止，运行进程（process）的底层**机制**（mechanism），例如上下文切换（context switch），应该已经清楚了；如果还不清楚，请回到前一两章，把这些机制的工作原理再读一遍。不过，我们还没有弄明白操作系统调度程序（scheduler）所采用的上层**策略**（policy）。下面就来做这件事：我们会介绍一系列**调度策略**（scheduling policy，有时也叫**调度规则**，discipline），它们是许多聪明又勤奋的人多年来陆续提出的。

事实上，调度的起源比计算机系统还早；最早的一些方法来自运筹管理领域，后来才被用到计算机上。这一点不足为奇：流水线以及人类的许多其他活动也都需要调度，其中有许多相同的考量，包括对效率近乎执着的追求。于是，我们的问题是：

关键问题：如何制定调度策略
思考调度策略，该从怎样的基本框架入手？有哪些关键的假设？哪些指标是重要的？最早的计算机系统采用过哪些基本方法？

7.1 工作负载假设

在探讨各种可能的策略之前，我们先对系统中运行的进程做一些简化假设，这些进程有时统称为**工作负载**（workload）。确定工作负载是制定策略的关键一环；对工作负载了解得越多，策略就能调校得越精细。

我们这里对工作负载所做的假设大多并不现实，不过这（暂时）没关系，因为我们会随着讨论的深入逐步放宽这些假设，最终得到一个可以称为……（戏剧性停顿）……**全面运转的调度规则**[1]的东西。

对于系统中运行的进程，我们做以下假设：

1. 每个作业的运行时间相同。
2. 所有作业同时到达。
3. 作业一旦开始，就一直运行到完成。
4. 所有作业都只使用 CPU（即不执行 I/O）。
5. 每个作业的运行时间是已知的。

[1]请用说“一个全面运转的死星”（A fully-operational Death Star）的语气读出来。

我们说过其中许多假设都不现实；不过，正如奥威尔的《动物庄园》[O45] 里有些动物比其他动物更平等，本章中也有些假设比其他假设更不现实。尤其是“每个作业的运行时间已知”这一条，也许会让你觉得别扭：这等于说调度程序无所不知；这样（大概）固然很好，但短期内恐怕不会实现。

7.2 调度指标

除了对工作负载做出假设，我们还需要一样东西来比较不同的调度策略：**调度指标**（scheduling metric）。指标就是我们用来衡量某样东西的标准，而在调度中，有好几种指标都说得通。

不过眼下，我们还是让生活简单一点，只用一个指标：**周转时间**（turnaround time）。作业的周转时间定义为作业完成的时刻减去作业到达系统的时刻。更形式化地说，周转时间 $T_{turnaround}$ 为：

$$T_{turnaround} = T_{completion} - T_{arrival} \tag{7.1}$$

由于我们假设所有作业同时到达，所以目前 $T_{arrival} = 0$，因而 $T_{turnaround} = T_{completion}$。随着我们放宽前面的假设，这一点会有所改变。

你应该注意到，周转时间是一个**性能**（performance）指标，它是本章关注的重点。另一个值得关注的指标是**公平性**（fairness），可以用 **Jain 公平性指数**（Jain's Fairness Index）[J91] 等方式来衡量。在调度中，性能和公平往往互相矛盾；例如，调度程序可能优化了性能，代价却是让几个作业无法运行，从而降低了公平性。这个难题告诉我们，生活并不总是完美的。

7.3 先进先出（FIFO）

我们能实现的最基本的算法叫作**先进先出**（First In, First Out，**FIFO**）调度，有时也叫**先来先服务**（First Come, First Served，**FCFS**）。

FIFO 有不少优点：它显然很简单，因而容易实现。而且，在我们的假设下，它的效果相当不错。

我们一起来看一个简单的例子。假设系统中大约同时（$T_{arrival} = 0$）到达了三个作业：A、B 和 C。由于 FIFO 总得先放一个作业，我们假设虽然它们同时到达，但 A 比 B 早到了一丁点，B 又比 C 早到了一丁点。再假设每个作业都运行 10 秒。这些作业的**平均周转时间**（average turnaround time）是多少？

从图 7.1 可以看出，A 在 10 秒时完成，B 在 20 秒时完成，C 在 30 秒时完成。因此，这三个作业的平均周转时间就是 $\frac{10+20+30}{3} = 20$。计算周转时间就这么简单。

现在我们放宽其中一个假设。具体来说，放宽假设 1，不再假设每个作业的运行时间相同。这时 FIFO 表现如何？你能构造出什么样的工作负载，让 FIFO 表现很差？

（继续读之前先想一想……接着想……想到了吗?!）

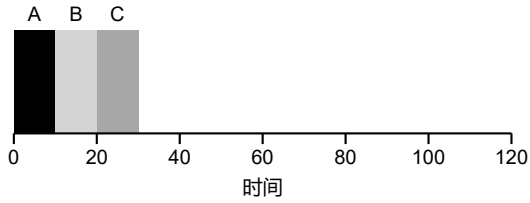


图 7.1：**FIFO 的简单例子**

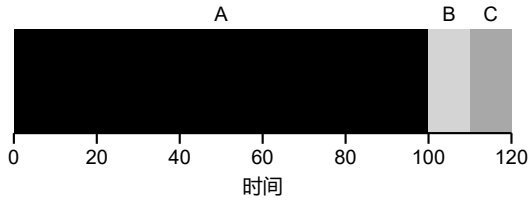


图 7.2：**FIFO 为什么没那么好**

想必你已经想出来了，不过以防万一，我们还是举个例子，看看长度不同的作业会给 FIFO 调度带来什么麻烦。具体来说，仍然假设有三个作业（A、B 和 C），但这次 A 运行 100 秒，而 B 和 C 各运行 10 秒。

从图 7.2 可以看到，作业 A 先完整地运行了 100 秒，B 和 C 才有机会运行。因此，系统的平均周转时间很高：足足 110 秒（$\frac{100+110+120}{3} = 110$），令人痛苦。

这个问题通常称为**护航效应**（convoy effect）[B+79]：一些耗时相对较短的资源使用者，排在了一个重量级资源使用者后面。这种调度场景也许会让你想起超市里只有一条结账队伍时的情形：看到排在你前面的人推着三车满满的货物，还掏出了支票簿，你会是什么心情；这得等上好一阵子了[2]。

那么该怎么办？面对作业运行时间长短不一的新现实，我们怎样才能设计出更好的算法？先想一想，再往下读。

提示：SJF 原则

最短作业优先（Shortest Job First）体现了一条通用的调度原则：只要系统在乎每位顾客（在我们这里就是作业）感受到的周转时间，就可以用它。想想你排过的任何一条队：如果相关的店家在乎顾客满意度，他们多半已经考虑到了 SJF。例如，超市通常设有“十件以下”通道，以免只买几样东西的顾客排在一家人后面干等，而那家人正为某场即将到来的核冬天囤货。

[2]这种情况下的建议做法：要么赶紧换一条队，要么深深地、慢慢地、放松地吸一口气。没错，吸气，呼气。没事的，别担心。

 © 2008–23, ARPACI-DUSSEAU

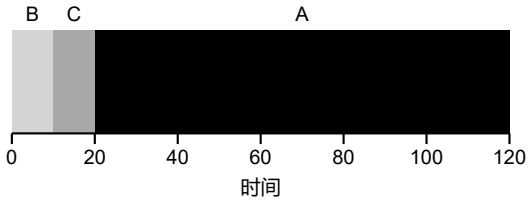


图 7.3：**SJF 的简单例子**

7.4 最短作业优先（SJF）

事实证明，一个非常简单的方法就能解决这个问题；实际上，这个想法是从运筹学 [C54, PV56] 借来，再用到计算机系统的作业调度上的。这种新的调度规则（scheduling discipline）叫作**最短作业优先**（Shortest Job First，SJF），名字应该很好记，因为它把策略说得相当完整：先运行最短的作业，再运行次短的，依此类推。

我们把上面的例子改用 SJF 作为调度策略。图 7.3 显示了运行 A、B、C 的结果。看了这张图，你应该就明白为什么 SJF 在平均周转时间上表现得好得多了。只是把 B 和 C 放在 A 之前运行，SJF 就把平均周转时间从 110 秒降到了 50 秒（$\frac{10+20+120}{3} = 50$），改善了一倍多。

事实上，在所有作业同时到达的假设下，我们可以证明 SJF 确实是一种**最优**（optimal）调度算法。不过，你上的是系统课，不是理论课或运筹学课；这里不许证明。

于是，我们借助 SJF 找到了一种不错的调度方法，但我们的假设仍然相当不现实。我们再放宽一个。具体来说，可以针对假设 2，改为假设作业可以在任何时刻到达，而不是同时到达。这会带来什么问题?

（再停下来想一想……你在想吗？加油，你能行的）

我们还是用一个例子来说明问题。这次假设 A 在 $t = 0$ 时到达，需要运行 100 秒；而 B 和 C 在 $t = 10$ 时到达，各需要运行 10 秒。如果用纯粹的 SJF，就会得到图 7.4 所示的调度。

从图中可以看出，虽然 B 和 C 在 A 之后不久就到达了，它们仍然不得不等到 A 完成，因此遭遇了同样的护航问题。这三个作业的平均周转时间是 103.33 秒（$\frac{100+(110-10)+(120-10)}{3}$）。调度程序能做些什么呢?

补充：抢占式调度程序

在过去的批处理（batch processing）时代，人们开发了一些**非抢占式**（non-preemptive）调度程序；这类系统会把每个作业都运行到完成，然后才考虑是否运行新的作业。几乎所有现代调度程序都是**抢占式**（preemptive）的，它们很乐意暂停一个进程的运行，转而运行另一个进程。这意味着调度程序用到了我们之前学过的机制；具体来说，调度程序可以执行**上下文切换**（context switch），暂时停下一个正在运行的进程，恢复（或启动）另一个进程。

 © 2008–23, ARPACI-DUSSEAU

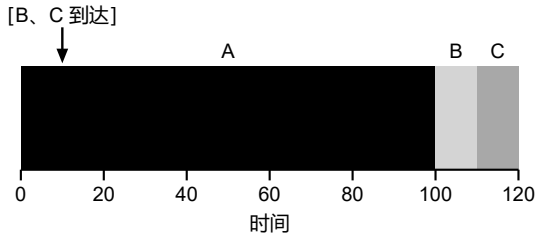


图 7.4：**B 和 C 晚到时的 SJF**

7.5 最短完成时间优先（STCF）

为了解决这个问题，我们需要放宽假设 3（作业必须运行到完成），那就放宽吧。我们还需要调度程序本身提供一些机制。根据前面对时钟中断（timer interrupt）和上下文切换的讨论，你可能已经猜到，B 和 C 到达时，调度程序当然可以有别的做法：它可以**抢占**（preempt）作业 A，决定运行另一个作业，以后再接着运行 A。按我们的定义，SJF 是一种**非抢占式**（non-preemptive）调度程序，因此会遇到上面描述的问题。

幸运的是，有一种调度程序恰好能做到这一点：给 SJF 加上抢占，这就是**最短完成时间优先**（Shortest Time-to-Completion First，**STCF**）调度程序，也叫**抢占式最短作业优先**（Preemptive Shortest Job First，**PSJF**）调度程序 [CK68]。每当有新作业进入系统，STCF 调度程序就判断剩下的作业（包括新作业）中谁的剩余时间最少，然后调度那个作业。因此，在我们的例子中，STCF 会抢占 A，把 B 和 C 运行到完成；只有它们都完成之后，才会调度 A 剩下的时间。图 7.5 展示了一个例子。

结果，平均周转时间大为改善：$\frac{(120-0)+(20-10)+(30-10)}{3} = 50$ 秒。和前面一样，在新的假设下，可以证明 STCF 是最优的；既然所有作业同时到达时 SJF 是最优的，你大概也能凭直觉看出 STCF 为什么是最优的。

7.6 新的指标：响应时间

因此，如果我们知道作业的长度，作业只使用 CPU，而且唯一的指标是周转时间，那么 STCF 就是一种很好的策略。事实上，对早期的一些批处理计算系统来说，这类调度算法也

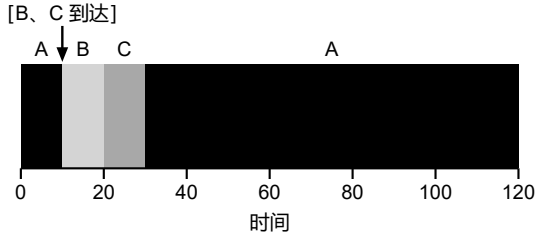


图 7.5：**STCF 的简单例子**

 © 2008–23, ARPACI-DUSSEAU

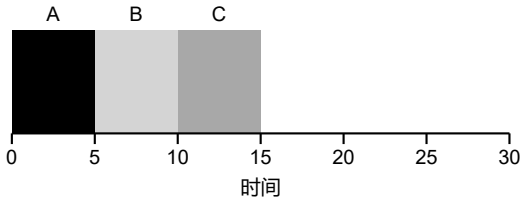


图 7.6：**又是 SJF（对响应时间不利）**

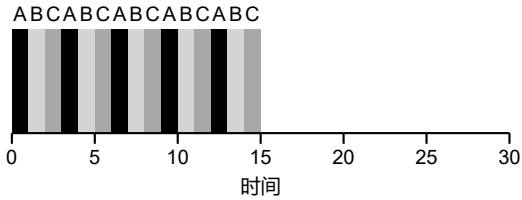


图 7.7：**时间片轮转（对响应时间有利）**

说得通。然而，分时（time sharing）机器的出现改变了这一切。现在，用户会坐在终端前，同时还要求系统有良好的交互性能。于是，一种新的指标诞生了：**响应时间**（response time）。

我们把响应时间定义为从作业到达系统到它第一次被调度的时间[3]。更形式化地说：

$$T_{response} = T_{firstrun} - T_{arrival} \tag{7.2}$$

例如，如果采用图 7.5 中的调度（A 在时刻 0 到达，B 和 C 在时刻 10 到达），各个作业的响应时间如下：作业 A 为 0，B 为 0，C 为 10（平均：3.33）。

你可能已经想到，STCF 及类似的调度规则（scheduling discipline）对响应时间并不怎么友好。例如，如果三个作业同时到达，第三个作业必须等前两个作业完整地运行完，才能被调度一次。这种方法虽然对周转时间很有利，对响应时间和交互性（interactivity）却相当糟糕。试想一下，你坐在终端前打字，却因为别的作业排在你前面，要等 10 秒才能看到系统的响应：可不怎么愉快。

于是，我们又面临一个问题：如何构建一个对响应时间敏感的调度程序？

7.7 时间片轮转

为了解决这个问题，我们引入一种新的调度算法，传统上称为**时间片轮转**（Round-Robin，**RR**）调度 [K64]。基本思路很简单：RR 不把作业运行到完成，而是让一个作业运行一个**时间片**（time slice，有时也叫**调度量子**，scheduling quantum），然后切换到运行队列（run queue）中的下一个作业。它反复这样做，直到所有作业都完成。因此，RR 有时也叫**时间分片**（time-slicing）。注意，时间片的长度必须是时钟中断周期的整数倍；因此，

[3]有些人的定义略有不同，例如还要算上作业产生某种“响应”之前的时间；我们的定义是其中最理想的情况，相当于假设作业一被调度就立即产生响应。

提示：平摊可以降低开销

某个操作有固定开销时，系统中常用一种通用技术：**平摊**（amortization）。少承担几次这笔开销（即少执行几次这个操作），系统的总开销就降下来了。例如，如果时间片设为 10 毫秒，而上下文切换的开销是 1 毫秒，那么大约 10% 的时间花在了上下文切换上，因而被浪费掉了。如果想*平摊*（amortize）这笔开销，可以加大时间片，比如设为 100 毫秒。这样一来，花在上下文切换上的时间不到 1%，时间分片（time-slicing）的开销就被**平摊**（amortize）掉了。

如果时钟每 10 毫秒中断（interrupt）一次，时间片就可以是 10 毫秒、20 毫秒或者其他任何 10 毫秒的倍数。

为了更详细地理解 RR，我们来看一个例子。假设三个作业 A、B、C 同时到达系统，而且都要运行 5 秒。SJF 调度程序会把每个作业都运行到完成，再运行下一个（图 7.6）。相比之下，时间片为 1 秒的 RR 会快速地在这几个作业之间轮转（图 7.7）。

RR 的平均响应时间是 $\frac{0+1+2}{3} = 1$；而 SJF 的平均响应时间是 $\frac{0+5+10}{3} = 5$。

可以看到，时间片的长度对 RR 至关重要。时间片越短，RR 在响应时间这一指标上的表现就越好。然而，时间片太短也有问题：上下文切换的开销会突然在整体性能中占据主导。因此，确定时间片长度对系统设计者来说是一种权衡：时间片要足够长，以便平摊（amortize）切换的开销；又不能太长，以免系统失去响应能力。

注意，上下文切换的开销并不只来自操作系统保存和恢复几个寄存器的操作。程序运行时，会在 CPU 缓存（cache）、TLB、分支预测器以及其他片上硬件中积累大量状态。切换到另一个作业时，这些状态会被清掉，换上与新作业相关的状态，这可能带来相当明显的性能开销 [MB91]。

因此，如果响应时间是我们唯一的指标，采用合理时间片的 RR 就是一个出色的调度程序。可是我们的老朋友周转时间呢？再来看上面的例子。A、B、C 的运行时间都是 5 秒，它们同时到达，调度程序是时间片为 1 秒（已经算长了）的 RR。从上图可以看出，A 在 13 秒时完成，B 在 14 秒时完成，C 在 15 秒时完成，平均为 14。相当糟糕！

所以毫不奇怪，如果以周转时间为指标，RR 确实是最*差*的策略之一。从直觉上讲，这也说得通：RR 每个作业只运行一小会儿就转到下一个，把每个作业的完成时间都拖到了尽可能晚。由于周转时间只关心作业何时完成，RR 几乎是最差的，在很多情况下甚至比简单的 FIFO 还差。

更一般地说，任何**公平**（fair）的策略（例如 RR），也就是在很短的时间尺度上把 CPU 平均分给各个活动进程的策略，在周转时间这类指标上都会表现不佳。这确实是一种固有的**权衡**（trade-off）：如果你愿意不公平，就可以先把较短的作业运行完，代价是响应时间；如果你看重公平，响应时间就会降低，代价是周转时间。这类权衡在系统中很常见；鱼与熊掌不可兼得[4]。

[4]这句英文俗语（You can't have your cake and eat it too）常常让人糊涂，因为它本该是"You can't *keep* your cake and eat it too"（蛋糕不能既留着又吃掉，这不是很明显吗？）。令人惊讶的是，维基百科上竟然有一页专门讲这句俗语；更令人惊讶的是，那一页读起来还挺有意思 [W15]。用意大利语说，就是不能 *Avere la botte piena e la moglie ubriaca*（既要酒桶满满，又要老婆喝醉）。

提示：重叠可以提高利用率

只要有可能，就让操作**重叠**（overlap）起来，以最大限度地提高系统的利用率（utilization）。重叠在许多不同的领域都很有用，包括执行磁盘 I/O 或向远程机器发送消息时；在这两种情况下，先启动操作，再切换去做其他工作，都是好主意，可以提高系统的整体利用率和效率。

我们已经设计了两类调度程序。第一类（SJF、STCF）优化周转时间，但对响应时间不利。第二类（RR）优化响应时间，但对周转时间不利。我们还有两个假设需要放宽：假设 4（作业不执行 I/O）和假设 5（每个作业的运行时间已知）。接下来就来处理这两个假设。

7.8 结合 I/O

首先，我们放宽假设 4：当然，所有程序都会执行 I/O。试想一个不接受任何输入的程序：它每次都会产生同样的输出。再试想一个没有输出的程序：它就像那句老话里在森林里倒下、却没人看见的树；它运行了也无关紧要。

当一个作业发起 I/O 请求时，调度程序显然要做一个决定，因为当前正在运行的作业在 I/O 期间不会使用 CPU；它被**阻塞**（blocked），等待 I/O 完成。如果 I/O 是发给硬盘驱动器（hard disk drive）的，进程可能会被阻塞几毫秒甚至更久，具体取决于驱动器当前的 I/O 负载。因此，调度程序这时大概应该在 CPU 上调度另一个作业。

I/O 完成时，调度程序也要做一个决定。这时会产生一个中断，操作系统开始运行，把发起 I/O 的进程从阻塞状态移回就绪（ready）状态。当然，它甚至可以决定在那时就运行这个作业。操作系统应该如何对待每个作业呢？

为了更好地理解这个问题，假设有两个作业 A 和 B，每个都需要 50 毫秒的 CPU 时间。不过，两者有一个明显的区别：A 运行 10 毫秒后就发起一次 I/O 请求（这里假设每次 I/O 耗时 10 毫秒），而 B 只是使用 CPU 50 毫秒，不执行 I/O。调度程序先运行 A，再运行 B（图 7.8）。

假设我们要构建一个 STCF 调度程序。A 被拆成了 5 个 10 毫秒的子作业，而 B 只是一次 50 毫秒的 CPU 需求，这样的调度程序该怎样考虑这一点呢？显然，只运行完一个作业再运行另一个，而不考虑如何把 I/O 纳入进来，是不太合理的。

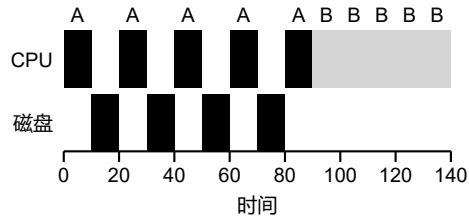


图 7.8：**资源利用不佳**

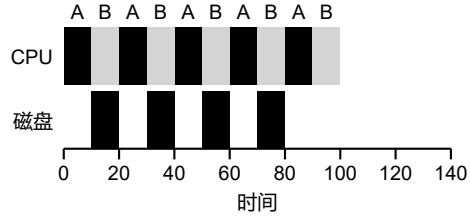


图 7.9：**重叠让资源得到更好的利用**

一种常见的做法是把 A 的每个 10 毫秒子作业都当作一个独立的作业。这样，系统启动时，它要选择的是调度一个 10 毫秒的 A，还是一个 50 毫秒的 B。用 STCF，选择很明确：选较短的那个，在这里就是 A。然后，A 的第一个子作业完成后，就只剩下 B，于是 B 开始运行。接着，A 提交了一个新的子作业，它抢占 B 并运行 10 毫秒。这样做就实现了**重叠**（overlap）：一个进程在使用 CPU 的同时，另一个进程在等待 I/O 完成；系统因此得到了更好的利用（见图 7.9）。

由此可见调度程序如何把 I/O 纳入考虑：把每个 CPU 突发（CPU burst）都当作一个作业，调度程序就能确保“交互式”的进程频繁地运行。在这些交互式作业执行 I/O 时，其他 CPU 密集型作业得以运行，从而更好地利用了处理器。

7.9 不再有先知

有了处理 I/O 的基本方法，我们来看最后一个假设：调度程序知道每个作业的长度。我们之前说过，这大概是我们能做出的最糟糕的假设。事实上，在通用操作系统中（比如我们关心的这些），操作系统通常对每个作业的长度知之甚少。那么，在没有这种先验（a priori）知识的情况下，我们如何构建一种表现得像 SJF/STCF 的方法呢？此外，我们如何把 RR 调度程序中的一些思想融合进来，让响应时间也相当好呢？

7.10 小结

我们介绍了调度背后的基本思想，并提出了两类方法。第一类运行剩余时间最短的作业，从而优化周转时间；第二类在所有作业之间交替运行，从而优化响应时间。一类的长处恰恰是另一类的短处，唉，这是系统中常见的固有权衡。我们还看到了如何把 I/O 纳入考虑，但仍然没有解决操作系统根本无法预知未来这个问题。很快我们就会看到如何克服这个问题：构建一个用最近的过去来预测未来的调度程序。这种调度程序叫作**多级反馈队列**（multi-level feedback queue），它是下一章的主题。

参考文献

[B+79] “The Convoy Phenomenon” by M. Blasgen, J. Gray, M. Mitoma, T. Price. ACM Operating Systems Review, 13:2, April 1979. *Perhaps the first reference to convoys, which occurs in databases as well as the OS.*

[C54] “Priority Assignment in Waiting Line Problems” by A. Cobham. Journal of Operations Research, 2:70, pages 70–76, 1954. *The pioneering paper on using an SJF approach in scheduling the repair of machines.*

[K64] “Analysis of a Time-Shared Processor” by Leonard Kleinrock. Naval Research Logistics Quarterly, 11:1, pages 59–73, March 1964. *May be the first reference to the round-robin scheduling algorithm; certainly one of the first analyses of said approach to scheduling a time-shared system.*

[CK68] “Computer Scheduling Methods and their Countermeasures” by Edward G. Coffman and Leonard Kleinrock. AFIPS ’68 (Spring), April 1968. *An excellent early introduction to and analysis of a number of basic scheduling disciplines.*

[J91] “The Art of Computer Systems Performance Analysis: Techniques for Experimental Design, Measurement, Simulation, and Modeling” by R. Jain. Interscience, New York, April 1991. *The standard text on computer systems measurement. A great reference for your library, for sure.*

[O45] “Animal Farm” by George Orwell. Secker and Warburg (London), 1945. *A great but depressing allegorical book about power and its corruptions. Some say it is a critique of Stalin and the pre-WWII Stalin era in the U.S.S.R; we say it’s a critique of pigs.*

[PV56] “Machine Repair as a Priority Waiting-Line Problem” by Thomas E. Phipps Jr., W. R. Van Voorhis. Operations Research, 4:1, pages 76–86, February 1956. *Follow-on work that generalizes the SJF approach to machine repair from Cobham’s original work; also postulates the utility of an STCF approach in such an environment. Specifically, “There are certain types of repair work, … involving much dismantling and covering the floor with nuts and bolts, which certainly should not be interrupted once undertaken; in other cases it would be inadvisable to continue work on a long job if one or more short ones became available (p.81).”*

[MB91] “The effect of context switches on cache performance” by Jeffrey C. Mogul, Anita Borg. ASPLOS, 1991. *A nice study on how cache performance can be affected by context switching; less of an issue in today’s systems where processors issue billions of instructions per second but context-switches still happen in the millisecond time range.*

[W15] “You can’t have your cake and eat it” by Authors: Unknown.. Wikipedia (as of December 2015). http://en.wikipedia.org/wiki/You_can’t_have_your_cake_and_eat_it. *The best part of this page is reading all the similar idioms from other languages. In Tamil, you can’t “have both the moustache and drink the soup.”*

 © 2008–23, ARPACI-DUSSEAU

作业（模拟）

程序 `scheduler.py` 可以让你看到，不同的调度程序在响应时间、周转时间和总等待时间等调度指标（scheduling metric）下表现如何。详情请见 README。

问题

1. 用 SJF 和 FIFO 调度程序运行三个长度为 200 的作业，计算响应时间和周转时间。
2. 现在做同样的事，但作业长度不同：100、200 和 300。
3. 再做同样的事，并且还要用时间片为 1 的 RR 调度程序算一遍。
4. 对于什么样的工作负载，SJF 的周转时间和 FIFO 相同?
5. 对于什么样的工作负载和量子长度（quantum length），SJF 的响应时间和 RR 相同?
6. 随着作业长度增加，SJF 的响应时间会怎样变化？你能用模拟器展示这个趋势吗?
7. 随着量子长度（quantum length）增加，RR 的响应时间会怎样变化? 给定 N 个作业，你能写出一个求最坏情况响应时间的公式吗?